

Exercise 01

Meet C

This is our first exercise. It's being given out on the first day of class, so it's called exercise 01. We'll keep the exercise numbers aligned with the lecture numbers. So, if we don't have an exercise for a particular day, the exercise numbering will just skip that day.

For this exercise, you get to fill in the missing parts of a simple C program. This will give you a chance to type in some C syntax yourself and to see some of the areas where the Java you already know will translate directly to C.

The program you're working on is called **arithmetic.c**; its job is to add up all the numbers from 1 to 10^9 (sum of an **arithmetic** series from one to one billion). You'll find a starting point for this program on the course homepage in Moodle, along with a copy of an expected output file.

You have some alternatives for how you can download the starter files you need for the exercise. You can download them from our Moodle page to your local machine using your web browser. Or, if you're working on one of the university Linux machines at `remote.eos.ncsu.edu`, you can copy all the starter files using a single **cp** command. Alternatively, if you're working on your own machine, you can still download the files from `remote.eos.ncsu.edu`; it just takes a slightly longer **scp** command.

Downloading Starter files from Moodle

To get started on this exercise, you can download the files you need from our course Moodle page, using your web browser. Be sure not to copy-and-paste the contents of the expected output file. Instead, download by right-clicking and choosing to save a copy of the file instead (this might vary a bit from browser to browser). If you copy-and-paste the contents of this file, you might accidentally change something in it (like you might miss the newline at the end of the line of output).

Copying Starter Files on `remote.eos.ncsu.edu`

Instead of downloading starter files using a web browser, if you're already logged in on one of the systems at `remote.eos.ncsu.edu`, you can enter the following shell commands to copy the needed files directly to your current directory. In addition to serving these files from our Moodle page, we've put copies in a shared directory that's available from university Linux machines.

```
cp /mnt/coe/workspace/csc/CSC230/ex/exercise01/* .
```

Don't miss this dot at the end.

We've put all the exercise files under a path like this, with a different two-digit number at the end of the word "exercise". If you like to use this technique for copying the exercise files, you should be able to use a similar command on all future exercises.

Copying Starter Files with SCP

Or, if you're working on an off-campus machine, you can still download the starter files using the command line. It just takes a longer command. The following command should download the all files you need. Just replace the word **unity** with your own unity ID, and you'll be prompted to type in your password after you enter the command.

```
scp 'dbsturgi@remote.eos.ncsu.edu:/mnt/coe/workspace/csc/CSC230/ex/exercise01/*' .
```

Dot at the end here

Completing the Assignment

In the source code of **arithmetic.c**, I've given you comments describing the code you need to add. In general, I'll mark places where you need to add code with a comment that looks like `// ...` or like `/* ... */`. You can delete these comments and replace them with the code you're supposed to write.

Once you've completed the program, you can build and run it with commands like the following. It may take a couple of seconds to complete execution, but it should print a line like the one below if it's working correctly:

```
$ gcc -Wall -std=c99 arithmetic.c -o arithmetic
$ ./arithmetic
500000000500000000
```

Callouts:

- We're using a dollar sign to show the shell prompt
- Here, we're compiling the program.
- Here, we're running it.
- Here's its output.

Exercise grading normally includes checking your program's output and checking its exit status to make sure it ran successfully. Most exercises provide a sample output file or two to help you check to make sure your output is exactly right. For this one, there's just a sample output file named **expected.txt**. If you put this file in the same directory as your program, you should be able to run commands like the following to check your program's exit status and make sure it prints exactly the right output:

```
$ ./arithmetic >| actual.txt
$ echo $?
0
$ diff expected.txt actual.txt
$
```

Callouts:

- Running the program with output saved to a file. (the `>|` says overwrite the output file)
- Checking the exit status. Good, it's zero. (have to do this right after you run)
- Comparing actual output with expected. Good. No differences reported.

The **arithmetic.c** program uses the long integer type. This is one example of where different compilers or different processors may behave differently. On the NCSU Linux machines, this type should give you a 64-bit integer for accumulating the sum. On some other systems, long will just be a 32-bit integer and you will get an answer that's incorrect. If you're getting a result like -243309312, it's probably because the representation for your sum isn't large enough. Be sure you're building and running your program on a common platform system.

Submitting your Solution

Once you have completed your program and you're happy with how it runs, submit your **arithmetic.c** source file to the **exercise_01** assignment in Moodle. Be sure your program has the right name, including the right capitalization and the **.c** filename extension at the end. Don't put your files in an archive or in a subdirectory; just submit the one **arithmetic.c** file.

Exercise Grading

Be sure to submit before **11:59 pm** on the due date. The exercises don't have a late submission window (although the projects do).

Each exercise is worth up to 10 points. After the exercise is graded, you'll get a score in the Moodle gradebook and a short text file with feedback about how your submission did when we tested it.

Keep in mind that this exercise will be graded automatically using a grading script. All the exercises are graded like that. To get full credit, you'll need to make sure your source file is named exactly the right thing, that it compiles cleanly on the common platform systems, prints exactly the right output when run and exits with successful exit status. The automated grading script can't tell the difference between important errors in your program's output and small, insignificant differences. If your program's output isn't what's expected, it will usually decide that your program's output is wrong. So, don't lose points for little mistakes; be sure to check your program's output against the provided expected output files.

This first exercise doesn't have any input, so it just includes one example expected output file. On future assignments, we'll usually provide one or two sample input files, along with corresponding expected output files. When we grade your programs, we'll usually run them on the provided example test cases along with other test cases that aren't published. We will sometimes check parts of your program's source code, its execution time or other behaviors to make sure you solved the problem the right way.